

An algorithm with linear expected running time for string editing with substitutions and substring reversals

Abdullah N. Arslan

Department of Computer Science, University of Vermont, Burlington, VT 05405, USA

Received 1 April 2007; received in revised form 7 November 2007; accepted 20 November 2007

Available online 4 December 2007

Communicated by F.Y.L. Chin

Abstract

The edit distance between given two strings X and Y is the minimum number of edit operations that transform X into Y without performing multiple operations that involve the same position. Ordinarily, string editing is based on character insert, delete, and substitute operations. Motivated from the facts that substring reversals are observed in genomic sequences, and it is not always possible to transform a given sequence X into a given sequence Y by reversals alone (e.g., X is all 0's, and Y is all 1's), Muthukrishnan and Sahinalp [S. Muthukrishnan, S.C. Sahinalp, Approximate nearest neighbors and sequence comparison with block operations, in: Proc. ACM Symposium on Theory of Computing (STOC), 2000, pp. 416–424; S. Muthukrishnan, S.C. Sahinalp, An improved algorithm for sequence comparison with block reversals, Theoretical Computer Science 321 (1) (2004) 95–101] considered a “simple” well-defined edit distance model in which the edit operations are: replace a character, and reverse and replace a substring. A substring of X can only be reversed if the reversal results in a match in the same position in Y . The cost of each character replacement and substring reversal is 1. The distance in this case is defined only when $|X| = |Y| = n$. There is an algorithm for computing the distance in this model with worst-case time complexity $O(n \log^2 n)$ [S. Muthukrishnan, S.C. Sahinalp, An improved algorithm for sequence comparison with block reversals, Theoretical Computer Science 321 (1) (2004) 95–101]. We present a dynamic programming algorithm with worst-case time complexity $O(n^2)$ but its expected running-time is $O(n)$. In our dynamic programming solution the weights of edit operations can vary for different substitutions, and the costs of reversals can be a function of the reversal-length.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Edit distance; Substring (block) reversal; Dynamic programming; Expected running time

1. Introduction

Computing the edit distance between two given strings X and Y is an important problem which has many applications such as estimating the evolutionary distances between biological sequences [11], and

constructing phylogenetic trees [5]. The edit distance between strings X and Y is the number of edit operations that transform X into Y [14]. Basic edit operations are: insert a symbol, delete a symbol, and substitute one symbol for another. Multiple operations in the same position are not allowed. Motivated from different applications, several studies have suggested extending the edit operations by including substring edits that reverse a substring, move or copy a substring from one location

E-mail address: aarslan@cs.uvm.edu.

to another, and delete a substring (see, for example, [12, 5]). In its general form when edit distance is computed allowing substring copy, delete, and move operations, the problem is NP-hard [8]. Shapira and Storer [13] give approximation algorithms for several cases of edit distance that use subsets of these substring operations. Arslan [1] proposes extending the basic edit model that includes operations (insert, delete, and substitute) at character level by adding substring reversal operation, and presents an algorithm for computing the edit distance in this model whose worst-case time complexity is $O(|X|^2|Y|)$ but that runs in $O(|X||Y|)$ expected time where $|X| \leq |Y|$. We note that computing edit distances in these extended models is different than the problem of sorting by reversals which uses reversals on a given permutation to generate a sorted order (see, for example, [4]).

Substring reversals are commonly observed in genomic sequences, and in multimedia data such as music scales [12]. Substring reversals in genomic sequences are usually a consequence of a large scale inter and intra chromosomal genomic duplications. Muthukrishnan and Sahinalp [9,10] study edit distance computation for an edit model that includes only character substitutions and substring reversals. In this case, the edit distance is defined when $|X| = |Y| = n$. The reversal of a substring is allowed only if the reversed substring matches a substring in the other string in the same position. There can be no multiple operations that involve the same position. The cost of each edit operation is 1. This is the “simplest” well-defined edit distance that involves substring reversals if we note that not all sequences can be transformed into all other sequences by reversals alone (e.g., X is all 0’s, and Y is all 1’s). For example, in this model string $X = \text{accaagagcg}$ can be edited to string $Y = \text{aggaacacgc}$ as follows: accaagagcg (no operation: a’s match) $\Rightarrow \text{accaagagcg}$ (replace c by g) $\Rightarrow \text{agcaagagcg}$ (reverse $caag$) $\Rightarrow \text{aggaacagcg}$ (no operation: a’s match) $\Rightarrow \text{aggaacagcg}$ (replace g by c) $\Rightarrow \text{aggaacacgc}$ (reverse cg) $\Rightarrow \text{aggaacacgc}$.

Muthukrishnan and Sahinalp introduced this edit model, and presented a $O(n \log^3 n)$ -time algorithm [9] where n is the common length of the strings X and Y . They later improved this complexity to $O(n \log^2 n)$ [10]. In this paper we consider their model. We first give a dynamic programming solution for computing the distance in this model. The worst-case time complexity of our solution is $O(n + |R|)$ where R is the set of all possible end position and length pairs each of which identifies a *reversible* substring in X with respect to Y . A substring of X is reversible with respect to Y if its reversal matches a substring of Y in the same position. We present an algo-

rithm that finds set R in time $O(n^2)$ in the worst-case but in $O(n)$ expected time based on our analysis that shows that the expected size of set R is $O(n)$.

The outline of this paper is as follows: in Section 2, we describe our notation, and present our dynamic programming solution for computing the edit distance in the model we consider. This solution assumes information about the end positions and lengths of all reversible substrings. In Section 3, we describe an algorithm that generates this information.

2. Edit distance with substitutions and substring reversals

Given two strings X and Y over an alphabet Σ where $|\Sigma| \geq 2$, the edit distance is the total number of operations needed to transform X into Y . A common model of string editing is based on insert, delete, and substitute operations at character level. There are other models (see for example [3,6,12,5,13]). In general, multiple operations in the same position in X are not allowed. This is necessary to obtain a metric that can be efficiently computed.

We denote by S_i the i th symbol of string S , and by $S_{i..j}$ the substring of S starting at position i , and ending at position j . We use $\tilde{S}$ to denote the *reverse* of string S , i.e. $\tilde{S} = S_{|S|} \dots S_2 S_1$.

In this paper, we consider the edit distance model introduced and studied first by Muthukrishnan and Sahinalp [9,10]. We say that substring $X_{(i-k+1)..i}$ of length k is *reversible* if $X_{(i-k+1)..i} = \tilde{Y}_{(i-k+1)..i}$. In this model, a character can be substituted for a character in X , and a substring of X can be reversed in a single step (if it is reversible). The assumption is that the string lengths are the same: $|X| = |Y| = n$, and multiple edit operations involving the same position in X are not allowed.

Let R_i be the lengths of reversible substrings ending at position i . More formally,

$$R_i = \{k \mid X_{(i-k+1)..i} = \tilde{Y}_{(i-k+1)..i}, \\ 1 \leq i \leq n, 1 \leq k \leq i\}.$$

If R_i contains $k > 0$ then by a single reversal, $Y_{(i-k+1)..i}$ can be obtained from $X_{(i-k+1)..i}$. We consider all possible reversals ending at position i for every i .

Let D_i denote the edit distance between $X_{1..i}$ and $Y_{1..i}$ in the model we study. We can compute D_i for all i , $1 \leq i \leq n$, as follows:

$$D_i = \min \left\{ D_{i-1} + s(X_i, Y_i), \min_{k \in R_i} D_{i-k} + f(k) \right\}, \quad (1)$$

where $D_0 = 0$, $s(X_i, Y_i)$ is the cost of replacing X_i by Y_i , and $f(k)$ is the cost of a reversal of length k .

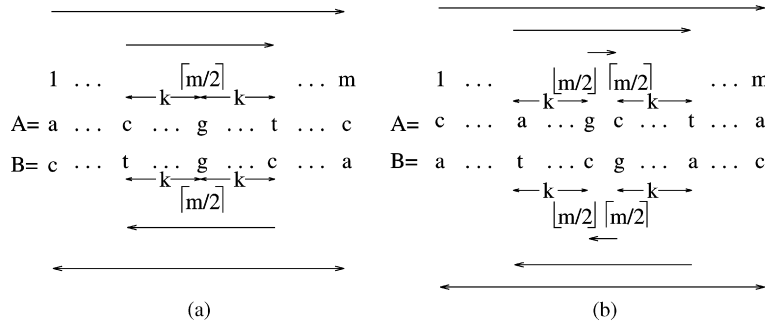


Fig. 1. Schematic description of Observation 1 for A and B where A is the reverse string of B . Parts (a) and (b) show, respectively, the cases when m is odd, and even. The start and end positions on the directed line segments in the figure identify the regions where there are reversible substrings. We include example symbols in A and B on selected positions to simplify the explanation of the observation.

The functions s and f are given. In the formulation of Muthukrishnan and Sahinalp [9,10], $s(X_i, Y_i)$ is the Hamming distance between X_i and Y_i , i.e. 0 when $X_i = Y_i$, and 1 otherwise, and $f = 1$. Note that in our formulation the substitution costs can vary, and the cost of reversals can be a function of the reversal-length.

Minimum distance D_i is achieved at position i by either adding $s(X_i, Y_i)$ (the cost of replacing X_i by Y_i ; in general, 0 if $X_i = Y_i$) to D_{i-1} , or by considering all possible reversals ending at position i of length k for all possible k and choosing an optimal one, i.e. a reversal of length k that yields the minimum total cost at position i when added to D_{i-k} .

Provided that R_i is given for all i , computing D_n using formulation (1) takes $O(n + |R|)$ time where $R = \bigcup_{i=1}^n R_i$. We note that if we calculate R_i 's by trying all possible reversal lengths k , this would require in total $\Omega(n^2)$ time in the worst case. We will give an algorithm that computes R_i for all i in time $O(n^2)$ in the worst case, but in $O(n)$ expected running time.

3. Computing all reversible substrings

We first give an observation which we use in our algorithm that finds lengths of all reversible substrings ending at position i for all positions i in X .

Observation 1. For any two strings A and B each of length $m \geq 1$ where $A = \bar{B}$,

- if m is odd, for all k , $0 \leq k \leq \lfloor m/2 \rfloor$,
 $A_{(\lfloor m/2 \rfloor - k) \dots (\lfloor m/2 \rfloor + k)} = \bar{B}_{(\lfloor m/2 \rfloor - k) \dots (\lfloor m/2 \rfloor + k)}$,
- if m is even, for all k , $0 \leq k < m/2$,
 $X_{(m/2 - k) \dots (m/2 + k + 1)} = \bar{Y}_{(m/2 - k) \dots (m/2 + k + 1)}$.

Observation 1 simply says that in a given reversal there is a sequence of shorter reversals centered at the

same position. To explain this more formally, we first state the following which we call the *reversal extension rule*: if $A_{i..j} = \bar{B}_{i..j}$, $A_{i-1} = \bar{B}_{j+1}$, and $A_{j+1} = \bar{B}_{i-1}$ then $A_{(i-1) \dots (j+1)} = \bar{B}_{(i-1) \dots (j+1)}$. We can interpret this rule as follows: if we first locate the initial reversible substring, which we call the *seed* (of length 1 if m is odd, and of length 2 if m is even) at the center position of A and B where $A = \bar{B}$, and then extend this reversal by applying the reversal extension rule then we obtain longer reversible substrings after each extension until the entire strings are covered. Fig. 1 schematically describes all such reversals. We include example symbols in selected positions in A and B to highlight that all symbols in A and B at equidistance from the center in opposite directions match given that $A = \bar{B}$.

For given two strings A , and B of length m where $A = \bar{B}$, if we find the seed and apply the reversal extension rule incrementally as long as new extensions are possible, we find all reversible substrings centered at this seed. We can stop as soon as the rule does not yield a new reversal because there will not be any longer reversal centered at this seed. If we do the same for all initial seeds that we find in any given two strings X and Y (not necessarily $X = \bar{Y}$), we can compute the lengths of all reversible substrings of X with respect to Y centered at all possible positions.

Corollary 1. For any two given strings X and Y , the maximum number of reversals centered at a given position i is less than one plus half of the length of the longest reversible substring centered at position i .

Fig. 2 shows our algorithm *FindAll* which computes and returns $R = \bigcup_{i=1}^n R[i]$ where $R[i]$ is the set of lengths of all reversible substrings of X with respect to Y ending at position i . We consider reversible substrings of odd length, and even length separately in Parts 1

```

Algorithm FindAll
/* Part 1: find all odd-length reversible substrings */
for i:=1 to n do
  if X[i]=Y[i] then M1[i]:=1 else M1[i]:=∅
for i:=1 to n do
  if 1∈M1[i] then {
    k:=1
    while ((i-k≥1 and i+k≤n) and (X[i-k]=Y[i+k] and X[i+k]=Y[i-k])) do
      {M1[i]:=M1[i]∪{2*k+1}; k:=k+1}
  }
/* Part 2: find all even-length reversible substrings */
for i:=1 to n-1 do
  if X[i]=Y[i+1] and X[i+1]=Y[i] then M2[i]:=2 else M2[i]:=∅
for i:=1 to n-1 do
  if 2∈M2[i] then {
    k:=2
    while ((i-k≥1 and i+k≤n) and (X[i-k+1]=Y[i+k] and X[i+k]=Y[i-k+1])) do
      {M2[i]:=M2[i]∪{2*k+2}; k:=k+1}
  }
/* Part 3: create R from M1 and M2 */
for i:=1 to n do R[i]:=∅
for i:=1 to n do {
  for every r∈M1[i] do
    {k:=(r-1)/2; R[i+k]:=R[i+k]∪{r}}
  for i:=1 to n-1 do
    for every r∈M2[i] do
      {k:=r/2-1; R[i+k+1]:=R[i+k+1]∪{r}}
}
return R

```

Fig. 2. Algorithm *FindAll* which finds all reversible substrings for all possible end-positions in X .

and 2 of the algorithm, respectively. In Part 3 we combine the results of Parts 1 and 2 to generate array R .

In Part 1, we first find all reversible substrings of odd length. We index reversals by their midpoints (seed positions). For odd-length reversals the seeds are positions of single symbols. We compute all reversals centered at these seeds, and we keep track of them in array M_1 . We start with examining each position i , $1 \leq i \leq n$, to identify the seeds for reversals of odd length. If X and Y match at position i then this means that there is a reversible substring of length 1 at position i , and it can potentially be a midpoint (seed) of larger reversals. We apply the reversal extension rule to see if the current reversible substring can be extended. If the answer is yes then we enlarge the reversal centered at this seed. We continue this process until no more extension is possible. By Observation 1 the last extension yields the longest reversible substring centered at seed position i . At this point, array $M_1[i]$ stores the lengths of all reversible substrings of odd-length whose seeds are at position i . We repeat these steps for all i , $1 \leq i \leq n$.

Part 2 of the algorithm is similar to Part 1. The seeds in this case are length-two reversible substrings. We calculate positions in reversible substrings relative to the position of the first symbol of the seed. After the execu-

tion of this part, $M_2[i]$ stores the lengths of all reversible substrings of even-length whose length-two seeds start at position i for all i , $1 \leq i \leq n-1$.

In the last part, Part 3, of the algorithm we compute R from M_1 and M_2 . From the seed positions i , and lengths of the reversible substrings in $M_1[i]$ and $M_2[i]$ we identify their end positions, and collect all reversible substring lengths (odd or even) in array $R[i]$. For every $r = 2k+1 \in M_1[i]$ there is a reversal ending at position $i+k$ of length r . Similarly, for every $r = 2k+2 \in M_2[i]$ there is a reversible substring ending at position $i+k+2$ of length r . Clearly set $R[i]$ obtained at the end of Part 3 is the set of all possible lengths of reversible substrings ending at position i in X for all i , $1 \leq i \leq n$.

Next we establish the time complexity of Algorithm *FindAll*. With every iteration of each of the while loops in Parts 1 and 2, a new reversible substring is found except for the last iteration. Therefore, the time spent in Parts 1 and 2 is $O(\sum_{i=1}^n C_i)$ where C_i is the number of reversible substrings whose seeds start at position i . We note that by Corollary 1, C_i is bounded from above by the length of the longest reversible substring whose seed starts at position i . Therefore, $C_i = O(n)$, $|R| = O(n^2)$, and Algorithm *FindAll* in the worst case takes $O(n^2)$

time. However, we show that the expected size of R is $O(n)$, and the algorithm runs in $O(n)$ expected time.

Let S be a randomly chosen string, and i be any position in S . Let Q_s be the probability that symbol s appears in position i in S , and Q be the maximum Q_s over all $s \in \Sigma$. In a uniform distribution, the probabilities are all equal, and $Q = 1/|\Sigma|$. We assume that $Q < 1$. This is true for all practical domains (e.g., see [2,7] for study of individual symbol-probabilities in various biological domains).

Let $P_{i,k}^{\text{odd}}$ be the probability that there is a reversible substring of length $2k+1$ centered at position i in X , i.e. $X_{(i-k)..(i+k)} = Y_{(i-k)..(i+k)}$ for all k , $0 \leq k \leq \lfloor n/2 \rfloor$. We can see that the probability for any two symbols from X and Y to match is at most Q , and therefore, $P_{i,k}^{\text{odd}} \leq Q^{2k+1}$.

Let $C_{i,k}^{\text{odd}}$ be a 0–1 random variable such that $C_{i,k}^{\text{odd}} = 1$ indicates that there is a reversible substring of length $2k+1$ centered at position i , otherwise, $C_{i,k}^{\text{odd}} = 0$. Define C_i^{odd} as a random variable that denotes the number of reversible substrings whose seeds are at position i . We note that although $P_{i,k}^{\text{odd}}$'s are not independent for all different values of i, k because of overlapping among substrings centered at the same position i , the linearity of expectation does not require independence, and therefore, the expected value of C_i^{odd} (i.e. the expected size of $M_1[i]$ in Algorithm *FindAll*) is

$$E(C_i^{\text{odd}}) \leq \sum_{k=0}^{\lfloor n/2 \rfloor} Q^{2k+1}. \quad (2)$$

Define C_i^{even} as the number of reversible substrings of even length whose seeds start at position i . By similar arguments in computing $E(C_i^{\text{odd}})$ we can see that the expected value of C_i^{even} (i.e. the expected size of $M_2[i]$ in Algorithm *FindAll*) is

$$E(C_i^{\text{even}}) \leq \sum_{k=0}^{n/2-1} Q^{2k+2}. \quad (3)$$

Define C as a random variable that denotes the number of reversible substrings of X with respect to Y . Then the expected value of C , using inequalities (2), and (3), is

$$\begin{aligned} E(C) &= E\left(\sum_{i=1}^n C_i^{\text{odd}}\right) + E\left(\sum_{i=1}^{n-1} C_i^{\text{even}}\right) \\ &< n \sum_{k=1}^{\infty} Q^k = n \frac{Q}{1-Q}. \end{aligned} \quad (4)$$

Inequality (4) implies that the expected size $|R|$ of the set R of the lengths of all possible reversible substrings of X with respect to Y is $O(n)$, and Algorithm

FindAll in Fig. 2 takes $O(n)$ expected time. We remark that the inequality (4) does not immediately yield a $O(n)$ time algorithm for finding set R . That is, even if there are $O(n)$ expected reversible substrings of X with respect to Y , we still need to find them efficiently to achieve $O(n)$ expected time. For example, if we naively try all possible reversal lengths at all possible end positions i , $1 \leq i \leq n$, then this takes $\Theta(n^2)$ time to find set R although $|R| = O(n)$ because with respect to the end positions i there can be multiple reversals whose lengths are not necessarily consecutive, and we need to check all possible $\Theta(n)$ reversal lengths for $\Theta(n)$ end positions performing $\Theta(n^2)$ comparisons in total. In our algorithm, we use the reversal extension rule: we examine reversals around their centers in increments of length two (one from each side), and we can stop as soon as the incremental extension fails. By Corollary 1, the time spent for finding the lengths of all reversals centered at any position i is linear in the number of reversals centered at position i . This way we avoid redundant computations, and achieve $O(n)$ expected time for computing set R .

The gap between the worst-case $O(n \log^2 n)$ time complexity obtained by Muthukrishnan and Sahinalp [10], and the achievement of $O(n)$ expected time by our algorithm is a motivation for future research for possible improvement of the worst-case time complexity of the problem.

References

- [1] A.N. Arslan, An algorithm for string edit distance allowing substring reversals, in: Proc. the 6th IEEE Symposium on Bioinformatics and Bioengineering (BIBE), Washington DC, October 16–18, 2006, pp. 121–126.
- [2] D.J. Brooks, J.R. Fresco, A.M. Lesk, M. Singh, Evolution of Amino acid frequencies in proteins over deep time: Inferred order of introduction of amino acids into the genetic code, *Mol. Biol. Evol.* 19 (10) (2002) 1645–1655.
- [3] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [4] S. Hannenhalli, P.A. Pevzner, Transforming cabbage into turnip: Polynomial algorithm for sorting signed permutations by reversals, *Journal of the ACM* 1 (46) (1999) 1–27.
- [5] M. Jackson, T. Strachan, G. Dover, *Human Genome Evolution*, Bios Scientific Publishers, 1996.
- [6] D.K. Kim, J.-S. Lee, K. Park, Y. Cho, Efficient algorithms for approximate string matching with swaps, *Journal of Complexity* 15 (1999) 128–147.
- [7] A.S. Kolaskar, B.V.B. Reddy, Complimentary DNA sequence data analysis of prokaryotic systems, *J. Biosci.* 7 (1) (March 1985) 49–59.
- [8] D. Lopresti, A. Tomkins, Block edit models for approximate string matching, *Theoretical Computer Science* 181 (1) (1997) 159–179.

- [9] S. Muthukrishnan, S.C. Sahinalp, Approximate nearest neighbors and sequence comparison with block operations, in: Proc. ACM Symposium on Theory of Computing (STOC), 2000, pp. 416–424.
- [10] S. Muthukrishnan, S.C. Sahinalp, An improved algorithm for sequence comparison with block reversals, *Theoretical Computer Science* 321 (1) (2004) 95–101.
- [11] P. Sellers, The theory and computation of evolutionary distances: Pattern recognition, *Journal of Algorithms* 1 (1980) 359–373.
- [12] D. Sankoff, J. Kruskal, *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison*, Addison-Wesley, Reading, MA, 1983.
- [13] D. Shapira, J.A. Storer, Large edit distance with multiple block operations, in: SPIRE 2003, in: *Lecture Notes in Comput. Sci.*, vol. 2857, Springer, 2003, pp. 369–377.
- [14] R.A. Wagner, M.J. Fisher, The string-to-string correction problem, *Journal of the ACM* 21 (1974) 168–173.